

DEVOPS MASTER COURSE

Volume 7C

Azure Databases & Data Platforms

Chapters 21-30

AZURE SQL | POSTGRESQL | COSMOS DB | MANAGED REDIS | DATA LAKES | DATA FACTORY |
EVENT HUBS | DATABRICKS | MIGRATION | GOVERNANCE

Production-grade transactional databases, distributed data, lakehouse analytics, streaming,
migration, security, labs, and interviews

2026 Edition

About This Part

Volume 7C develops the data layer of the Azure platform. Parts A and B established identity, governance, compute, networking, storage, hybrid connectivity, backup, and disaster recovery. Part C explains how production teams select, secure, operate, scale, migrate, and recover relational, NoSQL, cache, lakehouse, batch, and streaming data systems.

The engineering objective is not to memorize service names. It is to connect workload semantics to data models, consistency, latency, throughput, recovery objectives, access boundaries, observability, change management, and cost. Every chapter therefore combines architecture decisions with implementation, validation, failure analysis, operational evidence, and cleanup.

Safety, Security, and Cost Controls

- Use an approved sandbox subscription and region. Check service availability, quotas, pricing tiers, networking constraints, and feature status before provisioning.
- Create budgets, owner tags, environment tags, data-classification tags, and expiry dates before creating databases, private endpoints, integration runtimes, streaming capacity, or analytics workspaces.
- Use Microsoft Entra authentication and managed identities wherever supported. Do not place passwords, connection strings, access keys, SAS tokens, or service-principal secrets in source code or screenshots.
- Prefer private endpoints, network isolation, least-privilege RBAC, database-native authorization, encryption, auditing, and centralized secrets management.
- Use synthetic training data. Do not upload production personal, financial, health, government, or confidential information into labs.
- Delete idle databases, caches, integration runtimes, event-streaming capacity, clusters, private endpoints, backups, and diagnostic workspaces after validation.

Table of Contents

Chapter 21 Azure SQL Database and SQL Managed Instance Engineering

Chapter 22 Azure Database for PostgreSQL and MySQL Flexible Server

Chapter 23 Azure Cosmos DB and Globally Distributed NoSQL

Chapter 24 Azure Managed Redis and Application Caching

Chapter 25 Azure Data Lake Storage, OneLake, Fabric, and Synapse

Chapter 26 Azure Data Factory and Enterprise Data Integration

Chapter 27 Azure Event Hubs and Stream Analytics

Chapter 28 Azure Databricks and Lakehouse Engineering

Chapter 29 Azure Database and Data-Platform Migration Engineering

Chapter 30 Data Security, Governance, Resilience, and Enterprise Capstone

Navigation: Heading styles support Word and LibreOffice document-outline navigation and a field-based table of contents.

VOLUME 7C | CHAPTER 21

Chapter 21 - Azure SQL Database and SQL Managed Instance Engineering

Designing managed SQL platforms for availability, security, performance, recovery, and controlled change

Learning Objectives

- Distinguish Azure SQL Database, elastic pools, serverless compute, Hyperscale, and SQL Managed Instance.
- Design availability-zone, geo-replication, failover-group, backup-retention, and restore strategies from business objectives.
- Implement private connectivity, Microsoft Entra authentication, auditing, Defender controls, encryption, and least privilege.
- Diagnose connection, blocking, query-plan, storage, log, capacity, and failover problems with evidence.

21.1 Choose the SQL Operating Model Before the SKU

Azure SQL Database is a managed database service optimized for modern applications that can operate at the database boundary. SQL Managed Instance provides broader SQL Server instance compatibility, including instance-scoped capabilities and migration paths for applications that are difficult to refactor. The choice must be based on dependency analysis, feature compatibility, recovery requirements, network model, operational ownership, and cost rather than a preference for familiar names.

Service pattern	Best fit	Primary trade-off
Single Azure SQL database	Independent application database with clear ownership and predictable isolation.	Cross-database and instance-level assumptions may require redesign.
Elastic pool	Many databases with variable and noncoincident demand.	Noisy-neighbor risk must be controlled with per-database limits and monitoring.
Serverless SQL database	Intermittent noncritical workloads that tolerate resume latency.	Autoscaling and pause/resume behavior must match connection and latency expectations.
Hyperscale	Large or rapidly growing databases needing fast scale and read replicas.	Architecture, backup behavior, and cost model differ from conventional tiers.
SQL Managed Instance	High SQL Server compatibility, instance features, or migration with limited application change.	Larger network, cost, and provisioning footprint than a single database.

21.2 Availability, Backups, and Geo-DR

Built-in service availability is not a substitute for workload-level recovery design. Zone redundancy protects against eligible zonal failures within a region. Automated backups support point-in-time restore. Long-term retention supports compliance-driven retention. Active geo-replication creates readable secondaries per database, while failover groups coordinate failover and provide stable listener endpoints for groups of

databases or managed instances. The team must test application reconnection, DNS behavior, identity, firewall rules, secrets, jobs, and downstream dependencies during failover drills.

Engineering rule Define RPO and RTO first. Then choose zone redundancy, backup retention, geo-replication, failover groups, and application retry behavior that can demonstrably meet those objectives.

21.3 Security and Access Architecture

- Use Microsoft Entra authentication and managed identities for applications where supported; minimize SQL authentication.
- Use private endpoints and private DNS when public exposure is unnecessary. Disable broad firewall exceptions and avoid the “allow Azure services” shortcut as a default design.
- Use transparent data encryption, customer-managed keys where required, auditing, vulnerability assessment, Microsoft Defender for SQL, and immutable log retention.
- Separate Azure RBAC for resource management from database permissions for data access. Grant database roles to groups, not individual users, wherever possible.
- Protect schema migration with approvals, predeployment checks, backups, backward-compatible changes, and rollback or roll-forward procedures.

21.4 Performance Engineering and Safe Change

Performance investigations begin with workload symptoms and time correlation. Examine CPU, data IO, log IO, sessions, waits, blocking, deadlocks, query duration, plan changes, storage growth, and application retries. Query Store and automatic tuning can provide evidence, but they do not remove the need to understand data distribution, indexes, parameter sensitivity, transaction scope, and concurrency.

```
az sql db show -g rg-data -s sql-prod-01 -n appdb --query
'[{status:status,tier:sku.tier,capacity:sku.capacity,zoneRedundant:zoneRedundant}]'
az sql db list-usages -g rg-data -s sql-prod-01 -n appdb -o table
```

Troubleshooting Matrix

Symptom	Likely cause	Investigation	Corrective action
Application timeouts	Blocking, resource saturation, network path, exhausted connection pool, or long query.	Correlate application traces, SQL metrics, waits, blocking graph, connection counts, and network diagnostics.	Remove the bottleneck, bound transactions, tune queries/indexes, right-size, and configure retries with jitter.
Failover completes but app remains down	Connection string uses old host, DNS caching, unsupported driver retry, identity or firewall mismatch.	Test listener resolution, driver version, token acquisition, private DNS, and application logs.	Use failover-group endpoint, modern driver, retry policy, and identical security/network configuration.
Unexpected storage growth	Index maintenance, retained data, large objects, long transactions, or insufficient lifecycle process.	Review table/index size, retention jobs, log use, and growth history.	Implement retention, archive, maintenance, and capacity alerts; do not shrink repeatedly.
Deployment causes blocking	Schema change takes locks or application versions are incompatible.	Inspect active requests, lock waits, migration statements, and release timeline.	Use expand-and-contract migrations, online operations where supported, batching, and tested rollback.

Hands-on Lab 21: Deploy and Validate a Secure Azure SQL Database

Objective Create a training database with Microsoft Entra administration, private-connectivity planning, auditing, backup awareness, and repeatable validation.

Prerequisites

- Azure CLI authenticated to a sandbox subscription; a resource group and virtual network from earlier parts.
- A Microsoft Entra group for database administrators and synthetic sample data.
- A budget and cleanup tag applied before provisioning.

Implementation

1. Create a logical SQL server and database using a low-cost training configuration available in your region.
2. Configure a Microsoft Entra administrator and create a contained database user for a test group or managed identity.
3. Restrict network access. Where budget permits, create a private endpoint and private DNS link; otherwise document the production-private design.
4. Enable auditing to a protected destination and inspect diagnostic settings.
5. Create a table, load synthetic rows, capture a baseline query plan, and execute concurrent reads and writes.
6. Create a restore point by recording the current time, then test point-in-time restore to a new database.
7. Collect resource configuration, query evidence, audit evidence, and the restored database validation.

Validation Evidence

- Application identity can connect without a stored SQL password.
- Public connectivity is disabled or tightly restricted and the production private design is documented.
- Audit records and restore evidence are available.
- The restore creates a separate database and the original remains unchanged.

Cleanup

- Delete the restored database, original database, server, private endpoint, private DNS records, and test resource group after evidence capture.
- Confirm no retained backups, log destinations, or network resources continue to create unintended cost.

Stretch challenge Add a secondary region and failover group in a controlled environment, then perform a planned failover and measure application recovery time.

Interview Questions and Model Answers

1. When should a team choose SQL Managed Instance instead of Azure SQL Database?

Choose it when instance-level compatibility, SQL Agent, cross-database behavior, or migration constraints materially reduce application change. Validate every required feature and compare operational footprint and cost.

2. What is the difference between zone redundancy and a failover group?

Zone redundancy improves resilience within a region. A failover group provides cross-region disaster recovery and stable listener endpoints. They address different failure scopes.

3. Why is point-in-time restore not the same as high availability?

Restore recreates data from backups after corruption or deletion and takes time. High availability keeps a service running through eligible infrastructure failures.

4. How do you separate control-plane and data-plane authorization?

Use Azure RBAC for resource administration and database users/roles for SQL data access, with Microsoft Entra groups and managed identities connecting the two identity planes.

Chapter Summary

- Select the managed SQL operating model from compatibility and workload semantics.
- Design zone, backup, and geo-DR controls from tested RPO/RTO objectives.
- Use Entra authentication, private access, encryption, auditing, and separate data permissions.
- Treat query performance and schema change as observable production engineering disciplines.

VOLUME 7C | CHAPTER 22

Chapter 22 - Azure Database for PostgreSQL and MySQL Flexible Server

Operating managed open-source relational databases with HA, read scaling, secure connectivity, and engine-aware tuning

Learning Objectives

- Design PostgreSQL and MySQL Flexible Server deployments around engine behavior, availability, backups, and network isolation.
- Use high availability, read replicas, maintenance windows, parameter governance, and connection pooling appropriately.
- Implement Entra authentication where supported, TLS, private access, secrets controls, and database-native roles.
- Troubleshoot replication lag, connection exhaustion, vacuum or bloat, locks, slow queries, and failover behavior.

22.1 Managed Service Does Not Remove Engine Knowledge

Flexible Server manages infrastructure, patching capabilities, backups, and service integration, but the database team still owns schema design, SQL quality, transactions, connection behavior, indexes, data retention, extension governance, and application compatibility. PostgreSQL autovacuum, transaction ID age, statistics, and replication semantics remain important. MySQL buffer pools, transaction isolation, indexes, and connection patterns remain important.

Capability	PostgreSQL engineering focus	MySQL engineering focus
High availability	Synchronous standby placement, failover testing, connection retry.	Standby design, failover behavior, DNS and driver retry.
Read replicas	Read-only routing, replica lag, long queries, replication slots where applicable.	Read scaling, lag monitoring, read-after-write semantics.
Connection scale	PgBouncer or application pooling; avoid connection-per-request.	Application pools or proxy patterns; bound concurrency.
Maintenance	Version support, extensions, vacuum, statistics, parameter changes.	Version support, optimizer statistics, parameter and index maintenance.
Backup/restore	PITR, retention, geo-redundant backup options, logical export where required.	PITR, retention, export/import tests, logical consistency checks.

22.2 Availability, Replicas, and Maintenance

High availability creates a physically separate standby and synchronously protects committed transactions. Read replicas solve read-scaling or reporting needs but are not automatically equivalent to synchronous HA. Maintenance windows should align with business risk and include application retry testing. Major version

upgrades require extension and application compatibility assessment, performance baselines, rollback planning, and a rehearsal with production-like data.

22.3 Network and Identity Design

- Prefer private network access and private DNS for production servers. Document how CI/CD agents, administrators, applications, and monitoring reach the endpoint.
- Use TLS and verify certificates. Do not disable certificate validation to work around DNS or trust problems.
- Use Microsoft Entra authentication where the selected service and client support it; otherwise store generated credentials in Key Vault and rotate them.
- Grant database privileges through roles, separate schema ownership from application runtime, and prevent applications from using server administrator identities.
- Capture server logs, query statistics, audit data, and platform metrics to approved observability destinations.

22.4 Engine-Aware Troubleshooting

```
az postgres flexible-server show -g rg-data -n pg-prod-01 --query
'{state:state,version:version,ha:highAvailability.mode,storage:storage.storageSizeGb}'
az mysql flexible-server show -g rg-data -n mysql-prod-01 --query
'{state:state,version:version,sku:sku.name}'
```

Troubleshooting Matrix

Symptom	Likely cause	Investigation	Corrective action
Too many connections	Unbounded pools, leaked connections, idle sessions, or undersized compute.	Inspect server connection metrics and database session views; correlate with deployments.	Bound and reuse pools, add timeouts, fix leaks, and scale only after demand is understood.
PostgreSQL table bloat or rising transaction age	Long transactions, ineffective autovacuum, heavy churn, or insufficient maintenance resources.	Inspect long transactions, dead tuples, vacuum progress, and autovacuum settings.	End long transactions, tune autovacuum, reindex or rebuild carefully, and redesign churn patterns.
Replica lag	Heavy writes, long replica query, network or compute limits.	Review lag metrics, WAL/binlog generation, replica CPU/IO, and query workload.	Optimize writes, scale replica, cancel harmful reads, and route latency-sensitive reads appropriately.
Failover causes prolonged outage	Driver retries, DNS caching, pool behavior, or sessions not reconnecting.	Run a controlled failover and observe resolution, pool reset, and application error timeline.	Upgrade driver, implement bounded retry with jitter, reset pools, and reduce DNS caching assumptions.

Hands-on Lab 22: Operate a Highly Available Flexible Server

Objective Deploy a PostgreSQL or MySQL Flexible Server in a secure training topology and validate identity, backups, query visibility, and failover readiness.

Prerequisites

- A sandbox subscription with a delegated subnet or approved private-access design.
- A synthetic schema and workload generator.
- Budget approval for the selected temporary HA configuration.

Implementation

8. Create a Flexible Server with backups and a maintenance window; enable HA only where budget and region support permit.
9. Configure private connectivity and DNS, then verify connection from an approved client path.
10. Create separate owner, migration, read-only, and application roles.
11. Load synthetic data and capture baseline throughput, latency, connections, CPU, storage, and engine statistics.
12. Create a read replica if supported by the lab budget and measure replication lag under write load.
13. Test a restore to a new server and validate row counts and application connectivity.
14. Document the failover or restore runbook, including application retry, DNS, secrets, and monitoring checks.

Validation Evidence

- Application role cannot administer the server or unrelated schemas.
- Connection is encrypted and uses the approved private path.
- Baseline and replication or restore evidence is retained.
- The recovery runbook contains measurable RPO and RTO observations.

Cleanup

- Delete replicas, restored servers, private endpoints, DNS records, and the training server.
- Confirm backup retention and monitoring resources are removed or intentionally retained.

Stretch challenge Run a controlled connection-storm test, tune the application pool, and show how bounded concurrency improves latency without increasing database compute.

Interview Questions and Model Answers

1. Is a read replica a replacement for high availability?

No. A read replica is generally asynchronous and supports read scaling or DR scenarios. HA uses a standby model designed for automatic service failover and committed-data protection.

2. Why can a managed PostgreSQL service still experience bloat?

The service manages infrastructure, not application transaction discipline. Long transactions and heavy update/delete workloads can prevent vacuum from reclaiming space effectively.

3. What should be tested before a major engine upgrade?

Extension support, driver compatibility, SQL behavior, performance baselines, parameter changes, backup/restore, replication, maintenance duration, and rollback or fallback strategy.

4. Why is connection pooling essential?

Database processes and memory are finite. Reusing bounded connections prevents connection storms, reduces authentication overhead, and protects latency under load.

Chapter Summary

- Managed open-source databases still require engine-specific operational knowledge.
- HA, read replicas, backups, and maintenance solve different reliability needs.
- Private connectivity, TLS, identity, and database roles must be designed together.

VOLUME 7C | CHAPTER 23

Chapter 23 - Azure Cosmos DB and Globally Distributed NoSQL

Designing partitioned, elastic, multi-region data systems with explicit consistency and cost trade-offs

Learning Objectives

- Model Cosmos DB workloads using access patterns, partition keys, item size, indexing, throughput, and transaction boundaries.
- Choose consistency, single- or multi-region writes, and conflict handling from business semantics.
- Operate provisioned, autoscale, or serverless capacity with cost and hot-partition controls.
- Diagnose throttling, skew, high request units, latency, indexing, and regional failover behavior.

23.1 Partition Key Is an Architecture Decision

Cosmos DB distributes logical partitions across physical infrastructure. The partition key determines data locality, scalability, transaction boundaries, query fan-out, and hot-partition risk. A good key has high cardinality, spreads request and storage load, aligns with common queries, and preserves the atomic operations the application requires. A key chosen only because it is unique can still create inefficient cross-partition queries.

Decision	Questions to answer	Failure if ignored
Partition key	Which fields appear in reads/writes? Where are transactions required? Is load evenly distributed?	Hot partitions, fan-out queries, expensive migration.
Consistency	Does the user require read-your-writes, globally ordered reads, or eventual convergence?	Incorrect assumptions, unnecessary latency, or lower availability.
Throughput model	Steady demand, bursty demand, very small intermittent workload, or shared database capacity?	Throttling or persistent overprovisioning.
Index policy	Which paths are filtered, sorted, or excluded? Are composite indexes needed?	High write RU, failed order-by queries, expensive scans.
Region topology	Read regions, write regions, data residency, failover priorities, and conflict semantics?	Cross-region latency, conflict surprises, or noncompliance.

23.2 Consistency and Global Distribution

Cosmos DB provides five consistency levels: strong, bounded staleness, session, consistent prefix, and eventual. Session consistency is a common application default because it provides read-your-writes within a session while retaining good distribution characteristics. Strong consistency can add cross-region latency and constrains topology. Multi-region writes improve write availability and locality but require explicit conflict-resolution semantics and careful testing of business invariants.

Design warning Never choose consistency by label alone. Write down the exact user-visible anomaly that is acceptable or unacceptable, then test it across regions and failure scenarios.

23.3 Request Units, Indexing, and Cost

Every operation consumes request units based on item size, indexing, query shape, consistency, and operation type. A query returning few rows can still be expensive if it scans many partitions. Monitor normalized RU consumption, throttled requests, partition-level distribution, latency, item size, and query metrics. Treat HTTP 429 as a capacity and workload-shape signal; SDK retries help absorb transients but cannot correct a persistently hot key or inefficient query.

```
az cosmosdb show -g rg-data -n cosmos-prod-01 --query
'[{kind:kind, locations:locations[].locationName, consistency:consistencyPolicy.defaultConsistencyLevel}]'
az cosmosdb sql container throughput show -g rg-data -a cosmos-prod-01 -d appdb -n orders
```

23.4 Transactions, Change Feed, and Operational Patterns

- Transactions are scoped to a logical partition for standard transactional batches; model invariants accordingly.
- Use change feed for event-driven projections, integration, archival, and materialized views. Consumers must be idempotent and checkpointed.
- Use time-to-live for ephemeral data where deletion semantics and index cost are understood.
- Use private endpoints, managed identities or Entra-integrated access where supported, customer-managed keys when required, diagnostic settings, and continuous backup for critical workloads.
- Plan partition-key evolution early because changing it usually requires data movement to a new container.

Troubleshooting Matrix

Symptom	Likely cause	Investigation	Corrective action
Frequent 429 throttling	Provisioned throughput too low, burst exceeds autoscale ceiling, hot partition, or inefficient query.	Inspect normalized RU, partition key range metrics, query metrics, and retry-after values.	Fix key/query design, spread load, increase capacity deliberately, and use SDK retries with backoff.
High latency with low result count	Cross-partition fan-out, strong consistency, large item, or remote region.	Review query plan, contacted partitions, item size, consistency, and preferred regions.	Add targeted indexes, query by key, reduce item size, use appropriate consistency and regional routing.
Uneven storage or traffic	Low-cardinality or time-correlated partition key.	Compare per-partition storage and RU; replay representative traffic.	Introduce a better synthetic/hierarchical key or migrate to a redesigned container.
Unexpected conflict outcome	Multi-write conflict policy does not match business rule.	Review conflict feed, timestamps, custom resolver, and write-region sequence.	Define deterministic application conflict handling and test concurrent writes before production.

Hands-on Lab 23: Model and Load-Test a Cosmos DB Container

Objective Create a SQL API container, compare partition-key and query behavior, and collect RU, latency, and throttling evidence.

Prerequisites

- A sandbox Cosmos DB account or approved emulator for initial development.
- A synthetic order or telemetry dataset with multiple tenants and time periods.
- An SDK client that exposes request charge, diagnostics, and retry information.

Implementation

15. Define the top read, write, update, and transactional access patterns before creating the container.
16. Choose a candidate partition key and create the database and container with a low-cost capacity mode.
17. Load synthetic data with controlled distribution and record item size and request charge.
18. Run point reads, in-partition queries, and cross-partition queries; compare RU and latency.
19. Generate skewed traffic to one key and observe throttling or normalized RU distribution.
20. Enable change feed processing for an idempotent projection or archive target.
21. Document the final key choice, consistency level, throughput model, and recovery design.

Validation Evidence

- Point reads include id and partition key and have predictable request charge.
- Cross-partition query cost is measured rather than assumed.
- Hot-key behavior is visible in metrics and SDK diagnostics.
- Change-feed consumer can restart without duplicating business side effects.

Cleanup

- Delete the change-feed processor resources, container, database, and training account if dedicated.
- Confirm no diagnostic workspace or private endpoint remains unintentionally.

Stretch challenge Create a second container with an improved partition key, migrate the synthetic data, and present before/after RU and latency evidence.

Interview Questions and Model Answers

1. What makes a good Cosmos DB partition key?

It aligns with common access patterns and transaction boundaries, has high cardinality, distributes storage and request load, and avoids time- or tenant-based hot spots.

2. Why is session consistency commonly selected?

It provides read-your-writes and monotonic guarantees within a client session while preserving lower latency and higher availability than strong consistency across regions.

3. What does HTTP 429 mean?

The request was rate-limited. The SDK can retry using the server-provided delay, but persistent 429s require throughput, partition, query, or workload-shape correction.

4. Why is changing the partition key difficult?

It changes the physical distribution and transaction boundary. Existing data usually must be copied into a new container using a migration process.

Chapter Summary

- Partitioning, consistency, and throughput are application architecture decisions.
- Measure request units and partition distribution at workload level.
- Use change feed and transactions within their documented boundaries.
- Test regional failover and conflict behavior with real business invariants.

Partition Design Review

Review question	Required evidence
Will the key distribute both storage and requests?	Representative key-cardinality and per-partition RU/storage measurements.
Do transactions fit one logical partition?	Documented invariants and transactional-batch tests.
Can the workload tolerate the selected consistency?	Multi-client and multi-region anomaly tests.
Can the team migrate if the key becomes wrong?	Export/change-feed/backfill plan with validation and cutover criteria.

VOLUME 7C | CHAPTER 24

Chapter 24 - Azure Managed Redis and Application Caching

Engineering low-latency caches without turning volatile acceleration into an unowned data dependency

Learning Objectives

- Identify valid cache patterns, data ownership, freshness, invalidation, and failure behavior.
- Use Azure Managed Redis capabilities and plan migration from retiring Azure Cache for Redis offerings.
- Design network isolation, authentication, TLS, clustering, persistence, and geo options appropriately.
- Diagnose cache misses, eviction, hot keys, connection storms, latency, and failover effects.

24.1 A Cache Is a Consistency System

A cache accelerates access by keeping selected data closer to the application, but it introduces duplicate state and freshness decisions. The authoritative system of record must be explicit. Cache-aside, read-through, write-through, write-behind, session, distributed lock, rate-limit, and pub/sub patterns have different failure and consistency characteristics. The application must define behavior when the cache is unavailable, stale, partially populated, or evicting keys.

Pattern	Appropriate use	Primary risk
Cache-aside	Frequently read data that can be reloaded from a durable store.	Stale data and cache stampede after expiry.
Session cache	Short-lived web session state with designed expiration and recovery.	User disruption if session is not replicated or recoverable.
Distributed lock	Coordinating short critical sections with fencing and expiry.	Deadlock or split-brain if lock semantics are naïve.
Rate limiting	Fast counters and windows near request path.	Incorrect enforcement under failover or key skew.
Pub/sub	Ephemeral notifications where loss can be tolerated.	Messages are not durable queue semantics.
Primary datastore	Only for workloads whose durability and data model intentionally match the selected Redis offering.	Accidental data loss if cache assumptions and persistence are confused.

24.2 Service Lifecycle and Migration

Azure Managed Redis is the strategic managed Redis service. Existing Azure Cache for Redis estates must track Microsoft retirement timelines and migrate deliberately. Migration planning includes feature and tier mapping, network endpoints, client libraries, authentication, clustering, persistence, maintenance, geo design, capacity testing, and a cutover plan that avoids a cold-cache outage.

24.3 Capacity, Keys, and Client Engineering

- Reuse a bounded connection multiplexer or pool. Creating a connection per request causes handshake overhead and resource exhaustion.
- Use key naming, TTL, size limits, and ownership standards. Avoid unbounded collections and large values.
- Add TTL jitter and request coalescing to reduce stampedes when many keys expire together.
- Monitor memory, fragmentation, evictions, operations, server load, network, connections, latency, cache hit ratio, and hot keys.
- Design application fallback and circuit-breaker behavior. A cache outage should not automatically become a database-destroying retry storm.

```
az redisenterprise show -g rg-data -n redis-prod-01 --query
'[{sku:sku.name,provisioningState:provisioningState,hostName:hostName}]'
```

Troubleshooting Matrix

Symptom	Likely cause	Investigation	Corrective action
Latency spikes	Connection creation, large commands, hot key, server load, network path, or failover.	Inspect client diagnostics, command latency, connections, CPU, network, and key access distribution.	Reuse connections, reduce payload/command complexity, shard hot keys, and tune capacity.
High eviction rate	Working set exceeds memory or TTL policy is ineffective.	Review memory, fragmentation, key sizes, TTL distribution, and eviction policy.	Reduce item size, add TTL, remove unbounded keys, scale, or separate workloads.
Database overload after cache failure	No circuit breaker, synchronized misses, or cache stampede.	Correlate cache errors, database connections, query volume, and retry rates.	Use fallback budgets, request coalescing, stale-while-revalidate, jitter, and load shedding.
Low hit ratio	Wrong data cached, poor key design, TTL too short, or one-time access pattern.	Measure hits by feature and key family; compare cost of fill and reuse.	Cache only reusable expensive data; tune TTL and invalidate intentionally.

Hands-on Lab 24: Implement a Resilient Cache-Aside Pattern

Objective Deploy a temporary managed Redis cache and validate connection reuse, TTL, stampede protection, fallback, metrics, and cleanup.

Prerequisites

- A sandbox subscription with a small supported Azure Managed Redis configuration.
- A synthetic API and durable test datastore.
- A client library that supports TLS, retries, and connection reuse.

Implementation

22. Create the cache with private-access planning and approved authentication.
23. Implement cache-aside reads using deterministic keys, bounded TTL, and TTL jitter.
24. Instrument hit, miss, fill latency, fallback, connection, and datastore metrics.
25. Run a cold-cache load test and then a warm-cache test; compare datastore load and response latency.

26. Expire a popular key concurrently and demonstrate request coalescing or lock-based single fill.
27. Simulate cache unavailability and verify circuit breaking, load shedding, and bounded fallback.
28. Document the authoritative datastore, acceptable staleness, recovery behavior, and service-migration considerations.

Validation Evidence

- Connections remain bounded during load.
- Warm-cache latency and datastore load improve measurably.
- Concurrent expiry does not create an uncontrolled datastore spike.
- Cache outage behavior is visible and bounded.

Cleanup

- Delete the cache, private endpoints, DNS records, synthetic datastore, and test application resources.
- Confirm monitoring data retention matches the lab plan.

Stretch challenge Create a migration checklist from an Azure Cache for Redis tier to Azure Managed Redis, including feature mapping, capacity test, endpoint cutover, warm-up, and rollback.

Interview Questions and Model Answers

1. What is cache stampede?

Many callers miss or expire the same key and simultaneously regenerate it, overloading the system of record. Prevent it with coalescing, locks with fencing, TTL jitter, and stale responses where valid.

2. Why should an application reuse Redis connections?

Connection setup is expensive and creates server/client resource pressure. A long-lived thread-safe multiplexer or bounded pool improves throughput and stability.

3. Can Redis pub/sub replace a durable queue?

Not when guaranteed delivery, replay, or consumer recovery is required. Pub/sub is ephemeral and must be selected only when loss is acceptable.

4. What must be considered when migrating to Azure Managed Redis?

Feature compatibility, capacity, clustering, persistence, network, authentication, endpoint changes, client behavior, warm-up, cutover, monitoring, and rollback.

Chapter Summary

- Caching duplicates state and therefore requires explicit freshness and failure semantics.
- Azure Managed Redis is the strategic platform; migration must be engineered, not improvised.
- Client connection, key, TTL, memory, and stampede behavior determine reliability.
- A cache failure must not become an uncontrolled datastore outage.

VOLUME 7C | CHAPTER 25

Chapter 25 - Azure Data Lake Storage, OneLake, Fabric, and Synapse

Building governed analytical storage and selecting lakehouse, warehouse, and integrated analytics services

Learning Objectives

- Design ADLS Gen2 namespaces, zones, formats, partitioning, lifecycle, and access controls.
- Understand OneLake and Microsoft Fabric integration while recognizing Azure Synapse and dedicated Azure services.
- Choose lakehouse, warehouse, serverless query, Spark, and data-sharing patterns from workload requirements.
- Control small files, schema evolution, data quality, lineage, security, and cost.

25.1 Data Lake Architecture Is More Than a Storage Account

Azure Data Lake Storage Gen2 combines Blob Storage durability with a hierarchical namespace that supports directory semantics and analytics workloads. A production lake defines ownership, zones, naming, file formats, partitioning, retention, access boundaries, metadata, data quality, lineage, and workload isolation. Copying files into a container without these controls creates a data swamp rather than a platform.

Layer	Purpose	Engineering controls
Landing/raw	Preserve source records with minimal transformation.	Immutable arrival, source metadata, checksum, quarantine, retention.
Standardized/bronze	Normalize encoding, types, schema, and technical quality.	Schema version, deduplication, rejected-record handling, lineage.
Curated/silver	Create trusted domain-level entities and conformed data.	Business rules, quality tests, ownership, access classification.
Serving/gold	Optimize for BI, APIs, ML, or operational consumption.	SLAs, aggregates, semantic model, performance, product ownership.
Archive	Meet low-cost retention and legal requirements.	Lifecycle policy, immutability, retrieval test, deletion approval.

25.2 Formats, Partitioning, and the Small-File Problem

Columnar formats such as Parquet reduce scanned data and preserve analytical types. Delta Lake adds transaction-log semantics and schema/evolution capabilities for lakehouse workloads. Partitioning should enable pruning on common filters without creating excessive directories or skew. Thousands of tiny files increase metadata, listing, and task overhead; compaction and optimized writes are platform responsibilities.

25.3 OneLake, Fabric, and Synapse Service Choice

Microsoft Fabric provides an integrated SaaS analytics platform centered on OneLake, which exposes ADLS Gen2-compatible APIs and supports shared data across Fabric workloads. Azure Synapse Analytics continues to provide integrated SQL, Spark, pipelines, and workspace capabilities for existing and selected Azure architectures. Teams should evaluate governance, network isolation, workload engines, capacity model,

operational ownership, ecosystem fit, and migration cost instead of treating every analytics service as interchangeable.

Need	Likely starting point	Validate before commitment
Unified SaaS analytics and OneLake experience	Microsoft Fabric lakehouse, warehouse, Data Factory, and Power BI workloads.	Capacity, region, tenant governance, private-network needs, Git/CI support, and feature availability.
Azure-managed lake with independent engines	ADLS Gen2 with Databricks, Synapse, HDInsight alternatives, or custom compute.	Identity, networking, format standards, catalog, orchestration, and engine cost.
Serverless exploration of lake data	Serverless SQL or Fabric query experiences.	Supported formats, scan cost, concurrency, security, and latency.
Dedicated enterprise warehouse	Fabric Warehouse, Synapse dedicated SQL pool, or another selected warehouse.	Workload concurrency, loading, distribution, HA/DR, operations, and cost model.

25.4 Security, Governance, and Data Products

- Use managed identities, RBAC, and POSIX-style ACLs carefully. Avoid conflicting broad RBAC and granular ACL models that nobody can reason about.
- Use private endpoints and controlled egress where required; understand service-specific network behavior before promising isolation.
- Register assets, classifications, lineage, owners, and quality expectations in the enterprise catalog.
- Design domain data products with contracts, freshness, quality, discoverability, support, and lifecycle - not just folders.
- Use lifecycle rules, reserved capacity where appropriate, workload scheduling, scan controls, and observability to manage cost.

Troubleshooting Matrix

Symptom	Likely cause	Investigation	Corrective action
Analytical job is slow despite little data returned	Poor partition pruning, many small files, noncolumnar format, skew, or remote data path.	Inspect file count/size, query plan, bytes scanned, partition filters, and task distribution.	Compact, repartition, use columnar format, fix predicates, and colocate compute appropriately.
Users receive inconsistent access errors	RBAC, ACL, group propagation, token cache, or private DNS mismatch.	Trace identity, effective role, ACL chain, token audience, endpoint resolution, and network path.	Standardize group-based access, repair execute ACLs, refresh tokens, and correct private DNS.
Schema changes break consumers	No contract, uncontrolled inferred schema, or incompatible overwrite.	Compare source versions, table log/schema, pipeline deployment, and consumer assumptions.	Version contracts, validate compatibility, quarantine breaking data, and use additive evolution.
Storage cost rises unexpectedly	Duplicate zones, no lifecycle, temporary files, high transaction/list operations, or retained versions.	Analyze capacity by prefix, transaction metrics, versions, snapshots, and lifecycle execution.	Delete abandoned data, implement lifecycle, compact files, and assign cost ownership.

Hands-on Lab 25: Build a Governed Analytical Data Lake

Objective Create an ADLS Gen2 structure, ingest synthetic data, apply formats and permissions, and validate quality, lineage, lifecycle, and query efficiency.

Prerequisites

- A storage account with hierarchical namespace enabled in a sandbox subscription.
- A managed identity and Microsoft Entra groups for platform, producer, and consumer roles.
- Synthetic CSV or JSON source data with deliberate quality defects.

Implementation

29. Create landing, standardized, curated, serving, quarantine, and archive containers or directories with naming standards.
30. Apply group-based RBAC and required ACLs; verify a consumer cannot modify landing data.
31. Ingest source files with arrival metadata and checksums.
32. Convert valid records to Parquet or Delta, quarantine invalid records, and record quality metrics.
33. Partition by a field used in common filters and compare bytes scanned with and without partition pruning.
34. Generate many small files, measure the effect, then compact and compare again.
35. Configure a lifecycle rule for temporary or archive data and document the catalog/lineage model.

Validation Evidence

- Landing data is immutable to consumers and curated data is readable through approved identity.
- Quality failures are visible and do not silently enter curated data.
- Partition pruning and compaction reduce measured scan or task overhead.
- Lifecycle and ownership metadata are documented.

Cleanup

- Delete test datasets, storage resources, private endpoints, and temporary compute after evidence capture.
- Confirm soft-delete/versioning retention matches intended cleanup.

Stretch challenge Connect the lake to a Fabric or Synapse trial environment and compare governance, network, query, and cost implications of the integrated experience.

Interview Questions and Model Answers

1. What is the small-file problem?

A large number of tiny files increases metadata operations, planning, and task overhead. Compaction and optimized writes improve throughput and cost.

2. Why use Parquet for analytics?

It is columnar, typed, compressed, and supports predicate and column pruning, reducing IO for analytical queries.

3. What is OneLake?

It is Microsoft Fabric's unified logical data lake. It supports ADLS Gen2-compatible APIs and enables Fabric workloads to share data under a common governance experience.

4. How is a data product different from a folder?

A data product has an owner, contract, quality, freshness, security, documentation, support, and lifecycle. A folder is only storage organization.

Chapter Summary

- A governed lake defines layers, contracts, formats, partitioning, access, quality, and lifecycle.
- Fabric/OneLake, Synapse, Databricks, and independent Azure services must be selected from operational needs.
- Small files, schema drift, and permissions are recurring production risks.
- Data products require ownership and measurable service expectations.

Lakehouse Architecture Review

- Can every dataset be traced to an owner, source, contract, classification, and retention rule?
- Are file format, partitioning, clustering, and compaction decisions supported by measured query patterns?
- Are platform, producer, and consumer permissions understandable without individual exceptions?
- Can raw data be replayed to rebuild curated products after a faulty transformation?
- Are compute, storage, scan, and idle costs allocated to the responsible data product?

VOLUME 7C | CHAPTER 26

Chapter 26 - Azure Data Factory and Enterprise Data Integration

Orchestrating secure, observable, restartable batch and hybrid data movement

Learning Objectives

- Design Data Factory pipelines, datasets, linked services, triggers, parameters, and integration runtimes.
- Choose Azure, self-hosted, and managed-virtual-network integration runtime patterns.
- Build idempotent, restartable, metadata-driven pipelines with security and deployment controls.
- Diagnose copy, connectivity, schema, performance, concurrency, and retry failures.

26.1 Orchestration Is a State Machine

A pipeline coordinates work across systems with different reliability, security, and performance characteristics. Production orchestration records what should run, what actually ran, inputs, outputs, watermark or checkpoint, retries, timeout, owner, and recovery action. A green pipeline status is not proof that the expected business data arrived correctly.

Component	Responsibility	Production control
Linked service	Connection and authentication definition.	Managed identity or Key Vault reference; private network path.
Dataset	Logical data shape or location.	Parameterize paths and schema; avoid environment-specific duplication.
Activity	Copy, execute, transform, lookup, control flow, or external operation.	Timeout, retry, idempotency, output validation.
Pipeline	Orchestration and dependency graph.	Parameters, run correlation, checkpoints, failure routing.
Trigger	Schedule, tumbling window, or event initiation.	Backfill, concurrency, missed-run behavior, timezone.
Integration runtime	Compute/network bridge for movement and dispatch.	Capacity, placement, private endpoints, patching, HA for self-hosted IR.

26.2 Integration Runtime and Private Connectivity

Azure Integration Runtime handles cloud data movement and activity dispatch. A managed virtual network can isolate the runtime and use managed private endpoints for supported data stores. Self-hosted Integration Runtime connects to on-premises or restricted networks and must be treated as production infrastructure: multiple nodes for HA, controlled outbound access, patching, monitoring, certificate and key protection, capacity tests, and documented upgrade procedures.

26.3 Idempotency, Watermarks, and Metadata-Driven Design

- Use immutable source identifiers, watermarks, checkpoints, or change tracking so reruns do not duplicate or skip data.

- Write to staging and publish atomically where possible. Validate counts, hashes, quality, and freshness before promotion.
- Use metadata-driven loops for repeated source patterns, but keep schemas, ownership, SLAs, and exceptions explicit.
- Separate orchestration from heavy transformation when a specialized engine is more appropriate.
- Promote factory assets through source control and CI/CD with environment parameters, approval, and postdeployment validation.

```
az datafactory show -g rg-data -n adf-prod-01 --query
'{provisioningState:provisioningState,publicNetworkAccess:publicNetworkAccess}'
az datafactory pipeline list -g rg-data --factory-name adf-prod-01 -o table
```

Troubleshooting Matrix

Symptom	Likely cause	Investigation	Corrective action
Copy activity is slow	Small files, source throttling, sink throttling, low DIU/parallelism, network, or serialization.	Review activity details, throughput, source/sink metrics, file sizes, IR utilization, and parallel-copy settings.	Compact, tune partitioned copy, scale IR/DIU, schedule around limits, and optimize source/sink.
Self-hosted IR intermittently offline	Host sleep, proxy/firewall, resource exhaustion, certificate/service issue, or single node.	Check node status, Windows/service logs, outbound connectivity, CPU/memory, and version.	Use HA nodes, dedicated hosts, supported proxy rules, monitoring, and maintenance process.
Rerun duplicates data	No idempotent key, append-only sink, or watermark advanced too early.	Trace source window, checkpoint state, sink keys, and retry path.	Stage then merge/upsert, update checkpoint after validation, and use deterministic run identifiers.
Private endpoint pipeline cannot connect	Managed endpoint not approved, DNS, unsupported connector path, or wrong IR.	Inspect IR type, managed endpoint state, DNS, target firewall, and linked-service test.	Approve endpoint, repair DNS, choose correct IR, and document connector limitations.

Hands-on Lab 26: Build a Restartable Private Data Pipeline

Objective Create a parameterized Data Factory pipeline that copies incremental synthetic data through a secure runtime and proves rerun safety.

Prerequisites

- An Azure Data Factory instance and two approved synthetic data stores.
- A managed identity with least-privilege source and sink permissions.
- Private connectivity or a documented training substitute.

Implementation

36. Create linked services using managed identity or Key Vault references.
37. Create parameterized datasets and a pipeline that reads a watermark from a control table.
38. Copy only the new source interval into a staging path or table.
39. Validate row count, required columns, duplicate keys, and freshness before publishing.

40. Update the watermark only after successful validation and publication.
41. Force a failure after copy but before checkpoint, then rerun and prove no duplicate business rows are created.
42. Export or commit the factory artifacts and document environment-specific parameters.

Validation Evidence

- No credential appears in pipeline JSON or source control.
- A failed run can be replayed safely.
- Control-table state matches the last published data interval.
- Activity metrics and business-quality metrics are both retained.

Cleanup

- Stop triggers, delete pipeline resources, managed private endpoints, synthetic data, and the factory if dedicated.
- Remove self-hosted IR software from temporary lab hosts and revoke test permissions.

Stretch challenge Convert the pipeline to a metadata-driven pattern for three sources while keeping schema, owner, SLA, and error policy explicit for each source.

Interview Questions and Model Answers

1. What is an Integration Runtime?

It is the compute and network bridge used by Data Factory for data movement, activity dispatch, and SSIS execution. Azure, managed VNet, and self-hosted modes serve different connectivity needs.

2. How do you make a data pipeline idempotent?

Use deterministic keys, watermarks/checkpoints, staging, upsert or atomic publish, and advance state only after validation. A rerun must produce the same business result.

3. Why is a successful copy not sufficient evidence?

The wrong interval, schema, rows, or duplicates can still be copied successfully. Business-level count, quality, freshness, and reconciliation checks are required.

4. When is self-hosted IR appropriate?

When sources or sinks are reachable only from on-premises or restricted networks. It adds infrastructure ownership, HA, patching, capacity, and monitoring responsibilities.

Chapter Summary

- Data Factory pipelines must be restartable state machines with business validation.
- Integration Runtime choice determines network and operational ownership.
- Watermarks, staging, and atomic publish protect reruns.
- Pipeline code, parameters, identity, and evidence belong in controlled delivery workflows.

VOLUME 7C | CHAPTER 27

Chapter 27 - Azure Event Hubs and Stream Analytics

Engineering high-throughput event ingestion, schemas, consumer isolation, replay, and real-time processing

Learning Objectives

- Design Event Hubs namespaces, hubs, partitions, retention, throughput, consumer groups, and Capture.
- Use Schema Registry and compatible event contracts for producer-consumer evolution.
- Build Stream Analytics jobs with event-time semantics, windows, reference data, and late-event handling.
- Diagnose lag, throttling, partition imbalance, checkpoint, serialization, and out-of-order failures.

27.1 Event Streams Are Durable Logs, Not Work Queues

Event Hubs ingests high-volume ordered streams partitioned across a durable log. Consumers independently track offsets through consumer groups. Partitions are the unit of ordered processing and parallelism. Increasing partitions can improve parallel capacity but does not fix a poor partition key, slow consumer, or inefficient event size. A stream platform must define event identity, partition key, schema, retention, replay, ownership, and privacy before onboarding producers.

Design element	Purpose	Common mistake
Partition key	Keep related events ordered and distribute load.	Using tenant or device keys that create severe hot partitions.
Consumer group	Independent offset/checkpoint view for one logical application.	Creating one group per instance rather than per application.
Retention	Replay window after consumer outage or reprocessing need.	Choosing less retention than realistic recovery time.
Capture	Archive raw stream to storage for replay or analytics.	Assuming Capture replaces application checkpoints or validation.
Schema Registry	Central schema definition and compatibility workflow.	Changing fields without producer/consumer compatibility rules.
Throughput capacity	Ingress/egress and broker capacity.	Scaling broker capacity while a single partition remains hot.

27.2 Contracts, Idempotency, and Consumer Design

- Give each event a stable event ID, event type, schema version, source, subject, occurrence time, and trace/correlation context.
- Use Schema Registry for Avro or supported schema workflows and enforce compatibility in CI.
- Checkpoint after durable side effects, not before. Consumers must tolerate replay and duplicates.
- Use separate consumer groups for independent applications and scale processors no higher than useful partition parallelism.
- Protect sensitive fields through minimization, classification, encryption, access control, retention, and deletion procedures.

27.3 Stream Analytics and Event Time

Azure Stream Analytics processes streams using SQL-like queries, windows, joins, reference data, and output sinks. Correctness depends on event time, arrival time, watermark progression, late-event policy, out-of-order tolerance, partition alignment, and output idempotency. A job can be healthy from a service perspective while producing incorrect aggregates if event-time assumptions are wrong.

```
az eventhubs eventhub show -g rg-stream --namespace-name eh-prod-01 -n telemetry --query
'{partitions:partitionCount,retention:retentionDescription.retentionTimeInHours,status:status}'
az eventhubs namespace show -g rg-stream -n eh-prod-01 --query
'{sku:sku.name,capacity:sku.capacity,status:status}'
```

Troubleshooting Matrix

Symptom	Likely cause	Investigation	Corrective action
Consumer lag grows	Consumer too slow, insufficient partitions/instances, sink bottleneck, checkpoint issue, or hot partition.	Measure lag per partition, processing time, checkpoint age, consumer count, and sink latency.	Optimize processing/sink, scale within partition limits, repartition future events, and repair checkpoints.
Ingress throttling	Namespace capacity exceeded, event size, request rate, or quota.	Inspect throttled requests, incoming bytes/events, capacity, and producer retries.	Scale capacity or enable supported auto-inflate, batch sends, compress carefully, and reduce event size.
Duplicate output	At-least-once replay, checkpoint after failure, or nonidempotent sink.	Trace event ID, partition/offset, checkpoint, and sink write history.	Use deterministic idempotency keys, transactional sink pattern, and checkpoint after durable commit.
Incorrect window results	Late/out-of-order events, event-time parsing, timezone, or partition misalignment.	Compare event time, enqueue time, watermark, late-event metrics, and query settings.	Normalize timestamps, tune lateness policy, align partitioning, and test delayed-event fixtures.

Hands-on Lab 27: Build a Governed Real-Time Telemetry Pipeline

Objective Publish schema-controlled events, process them with independent consumers, archive them, and validate replay and late-event behavior.

Prerequisites

- An Event Hubs namespace and storage account in a sandbox subscription.
- A producer and consumer SDK with Entra/managed-identity support.
- Synthetic device or order events with controlled timestamps and duplicates.

Implementation

43. Create an event hub with multiple partitions, a retention period, and two consumer groups.
44. Create a schema group and register an event schema; add compatibility checks to the producer build.

45. Publish events using a partition key and include event ID, schema version, occurrence time, and trace ID.
46. Run two independent consumers and record checkpoint/lag behavior.
47. Enable Capture to ADLS or Blob Storage and verify archived event files.
48. Inject duplicate, delayed, and out-of-order events; prove consumer idempotency and window policy.
49. Stop a consumer, allow lag to build, restart, and measure catch-up time against retention.

Validation Evidence

- Events for one entity remain ordered within their partition.
- Independent consumers do not share checkpoints.
- Duplicate events do not create duplicate business outcomes.
- Archived events can be replayed and delayed-event behavior is documented.

Cleanup

- Delete Stream Analytics jobs if used, event hub, namespace, schema groups, Capture data, and temporary storage.
- Revoke producer/consumer roles and remove private endpoints.

Stretch challenge Create a partition-key comparison test that demonstrates throughput and ordering differences between random, tenant, and compound keys.

Interview Questions and Model Answers

1. What determines maximum consumer parallelism for one consumer group?

Useful parallelism is bounded by the partition count because one partition is actively processed by one processor instance in a coordinated consumer group.

2. Why are duplicate events normal?

Event processors generally use at-least-once delivery. Failures can occur after a side effect but before checkpoint, so replay is expected and consumers must be idempotent.

3. What is Schema Registry used for?

It centralizes event schemas and supports producer-consumer serialization and governance without embedding full schemas in every message.

4. What is the difference between event time and enqueue time?

Event time is when the business event occurred; enqueue time is when the broker received it. Window correctness often depends on event time plus late/out-of-order policy.

Chapter Summary

- Partitions govern ordering and parallelism; consumer groups isolate applications.
- Contracts, IDs, checkpoints, and idempotency make replay safe.
- Capture preserves raw events but does not replace consumer state.
- Real-time correctness requires explicit event-time and late-arrival semantics.

VOLUME 7C | CHAPTER 28

Chapter 28 - Azure Databricks and Lakehouse Engineering

Operating Spark, Delta Lake, Unity Catalog, jobs, clusters, quality, and cost as a governed data platform

Learning Objectives

- Design Azure Databricks workspaces, compute, jobs, Delta tables, and environment separation.
- Use Unity Catalog for centralized access, lineage, auditing, discovery, and data/AI governance.
- Engineer reliable batch and streaming pipelines with medallion patterns, checkpoints, and data-quality expectations.
- Diagnose skew, shuffle, small files, cluster startup, memory, autoscaling, and cost problems.

28.1 Lakehouse Platform Boundaries

Azure Databricks combines managed Spark and lakehouse capabilities with Azure identity, networking, storage, and governance integrations. The platform team defines workspace topology, network model, compute policies, catalogs, external locations, credentials, secret boundaries, logging, package controls, and CI/CD. Data teams own data products, transformations, quality, tests, and service expectations within those guardrails.

Platform area	Enterprise standard	Evidence
Catalog and access	Unity Catalog, groups, least privilege, workspace bindings, row/column controls where required.	Privilege export, lineage, audit logs, access reviews.
Compute	Cluster policies, approved runtimes, job/serverless compute where appropriate, autoscaling bounds, termination.	Policy compliance, utilization, cost by workload.
Code delivery	Repos or source-controlled bundles, automated tests, environment configuration, approvals.	Commit, build, deployment, and job-run correlation.
Data reliability	Delta transactions, schema enforcement, expectations, quarantine, checkpoints, recovery tests.	Quality metrics, table history, replay evidence.
Networking	Private connectivity and controlled egress according to workspace/service capabilities.	DNS, route, endpoint, firewall, and access validation.

28.2 Delta Lake and Reliable Pipelines

Delta Lake adds a transaction log over data files, enabling ACID transactions, schema enforcement, time travel, merges, and reliable batch/streaming convergence. These capabilities do not automatically create good data models. Teams still need stable keys, deduplication, late-data policy, partition and clustering strategy, compaction, retention, vacuum safety, and controlled schema evolution.

28.3 Unity Catalog and Governance

Unity Catalog provides centralized privileges, discovery, lineage, auditing, data classification, quality monitoring, and sharing across supported workspaces. Production governance uses Microsoft Entra groups, catalog/schema/table ownership, external locations, storage credentials or managed access, environment boundaries, and periodic entitlement review. Workspace-local secrets and unmanaged mounts should not become hidden alternatives to catalog governance.

28.4 Spark Performance and Cost

- Measure input size, file count, task time, shuffle read/write, skew, spills, executor memory, and output size before changing cluster size.
- Use broadcast joins only when the smaller side is safely bounded; avoid collecting large data to the driver.
- Compact small files, optimize layout for dominant access patterns, and avoid excessive partition cardinality.
- Use job clusters or serverless options where appropriate, cluster policies, autoscaling bounds, and automatic termination.
- Attribute cost to jobs, teams, and data products using tags/system tables and investigate idle or low-utilization compute.

```
databricks jobs list
databricks clusters list
# Example Spark SQL evidence
DESCRIBE HISTORY catalog.schema.orders;
```

Troubleshooting Matrix

Symptom	Likely cause	Investigation	Corrective action
One stage runs much longer than others	Data skew, uneven partitions, expensive UDF, or slow external dependency.	Inspect Spark UI task-duration distribution, shuffle size, partition keys, and query plan.	Repartition or salt skewed keys, use built-in functions, optimize join, and remove external calls.
Executor out of memory	Large partition, collect/cache misuse, join explosion, or insufficient executor memory.	Review failed task, GC, spill, partition size, cache, and plan cardinality.	Reduce partition size, remove unnecessary cache/collect, fix join, and size compute after code correction.
Too many small files	Frequent small writes, overpartitioning, streaming microbatches, or no compaction.	Count files and sizes by table/partition; inspect write frequency.	Use optimized writes/compaction, adjust trigger, and simplify partitioning.
Unexpected data access	Broad group privilege, ownership inheritance, unmanaged storage path, or workspace-local secret.	Review Unity Catalog grants, lineage, audit logs, external locations, and storage ACLs.	Revoke broad access, bind workspaces, centralize locations, rotate secrets, and add access reviews.

Hands-on Lab 28: Build a Governed Delta Lakehouse Pipeline

Objective Create a small medallion pipeline with Unity Catalog, Delta tables, quality checks, job deployment, and performance evidence.

Prerequisites

- An approved Azure Databricks workspace with Unity Catalog enabled.
- ADLS Gen2 or managed storage and Microsoft Entra groups.
- Synthetic orders and customers data containing duplicates, late records, and invalid fields.

Implementation

50. Create a catalog and schemas for bronze, silver, and gold with group-based privileges.
51. Ingest raw data into a Delta bronze table with source metadata.
52. Create a silver transformation that deduplicates, validates required fields, and quarantines failures.
53. Create a gold aggregate or serving table with a documented grain and owner.
54. Implement incremental processing with a checkpoint or change-data mechanism.
55. Generate a skewed join, inspect the Spark UI, apply a correction, and compare task distribution.
56. Deploy the workflow as a job from source control and collect lineage, quality, runtime, and cost evidence.

Validation Evidence

- Users can access only the catalog objects required by their group.
- Invalid records are measurable and quarantined.
- Rerunning the job does not duplicate silver or gold records.
- Performance correction is supported by Spark UI before/after evidence.

Cleanup

- Delete jobs, clusters, notebooks/repos if temporary, catalogs/schemas/tables, and training storage.
- Confirm compute has terminated and no cluster policy or private endpoint was unintentionally left behind.

Stretch challenge Add a streaming source from Event Hubs and prove exactly-once table results through checkpoint restart and duplicate-event injection.

Interview Questions and Model Answers

1. What does Delta Lake add to a data lake?

A transaction log and table protocol that enable ACID transactions, schema controls, version history, merge, and reliable batch/streaming operations over files.

2. What is Unity Catalog?

Azure Databricks' unified governance layer for data and AI assets, providing centralized access control, discovery, lineage, auditing, classification, and sharing.

3. How do you diagnose Spark skew?

Use the Spark UI to compare task durations and shuffle sizes, inspect key distribution and the physical plan, then correct partitioning, joins, or skewed keys.

4. Why prefer job clusters for scheduled workloads?

They provide isolated, reproducible compute that starts for the job and terminates afterward, reducing drift and idle cost compared with shared long-running clusters.

Chapter Summary

- Azure Databricks requires platform guardrails plus data-product ownership.
- Delta Lake improves transactional reliability but not data-model quality automatically.
- Unity Catalog centralizes governance and evidence.
- Spark tuning starts with plans, tasks, shuffle, skew, files, and utilization - not random cluster scaling.

VOLUME 7C | CHAPTER 29

Chapter 29 - Azure Database and Data-Platform Migration Engineering

Assessing compatibility, reducing downtime, validating data, and cutting over with evidence and rollback

Learning Objectives

- Classify migrations as rehost, replatform, refactor, or data-product redesign.
- Use assessment and migration tools such as Azure Migrate, Database Migration Service, engine tools, Data Factory, and offline transfer appropriately.
- Design online and offline migration waves with synchronization, validation, cutover, and rollback.
- Control identity, network, schema, performance, data quality, and organizational risks.

29.1 Migration Is a Business Change Program

A database migration changes latency, identity, network paths, operational tooling, failure behavior, licensing, backup, monitoring, and team responsibilities. The first deliverable is an inventory and dependency map, not a migration command. Classify source engines, versions, sizes, growth, change rate, downtime tolerance, features, extensions, jobs, integrations, compliance, owners, and business criticality.

Migration mode	Use when	Core controls
Offline backup/restore or export/import	Downtime is acceptable and data volume fits the window.	Timed rehearsal, checksum/count validation, DNS/connection cutover, rollback snapshot.
Online continuous replication	Downtime must be minimized and source/target support ongoing sync.	Lag monitoring, DDL handling, cutover freeze, final sync, failback decision.
Dual write	Application can safely write both systems during transition.	Idempotency, ordering, conflict reconciliation, data comparison, exit plan.
Bulk data movement	Analytics/history migration where files or tables can move in batches.	Manifest, checksums, partitioning, retries, reconciliation, secure transfer.
Rebuild data products	Legacy warehouse/lake semantics are being redesigned.	Contract mapping, lineage, quality baselines, parallel run, business sign-off.

29.2 Assessment and Wave Planning

- Run feature and compatibility assessment; inventory unsupported data types, extensions, collations, jobs, linked servers, stored procedures, and client drivers.
- Baseline throughput, latency, concurrency, storage, growth, maintenance, and batch windows on the source.
- Create migration waves by dependency and criticality, starting with representative low-risk workloads rather than only the easiest or hardest.
- Build the target landing zone, identity, network, logging, backup, security, and capacity before data movement.

- Define entry/exit criteria, decision owners, communication, freeze window, rollback trigger, and evidence package for each wave.

29.3 Validation Is More Than Row Count

Validation combines structural, data, functional, performance, security, and operational checks. Compare schemas, constraints, row counts, checksums or hashes, aggregates, nulls, referential integrity, sampled records, and business reconciliations. Then test applications, reports, batch jobs, identity, audit, backup, restore, failover, observability, and cost. A migration is not complete until the new system can be operated and recovered.

```
az datamigration service list -g rg-migration -o table
az datamigration sql-service list -g rg-migration -o table
az migrate project list -o table
```

Troubleshooting Matrix

Symptom	Likely cause	Investigation	Corrective action
Online migration lag will not converge	Source write rate exceeds apply rate, large transactions, network, target indexes, or unsupported changes.	Measure source change rate, replication throughput, transaction size, network, and target waits.	Scale/tune target, reduce batch impact, avoid large transactions, fix network, or choose a longer freeze/offline cutover.
Row counts match but application fails	Collation, timezone, identity, sequence, permissions, driver, or stored-procedure behavior differs.	Run functional tests, compare configuration, inspect errors, and trace representative transactions.	Correct compatibility, regenerate sequences, migrate permissions, update drivers, and retest.
Cutover exceeds window	Underestimated final sync, DNS/client cache, deployment, or validation duration.	Compare rehearsal timeline with actual lag and each cutover step.	Abort on preagreed threshold, restore source write access, and improve automation/rehearsal.
Target cost or performance is poor	Wrong tier, data model, indexes, partitioning, workload schedule, or service choice.	Compare baseline and target workload metrics and cost by component.	Tune schema/workload, resize, schedule, or reconsider target architecture before full migration.

Hands-on Lab 29: Plan and Rehearse a Low-Downtime Database Migration

Objective Assess a synthetic source database, migrate it to an Azure managed target, validate it, and execute a timed rehearsal with rollback criteria.

Prerequisites

- A synthetic source database on a local VM or temporary Azure VM.
- A target Azure SQL, PostgreSQL, or MySQL service in a sandbox subscription.
- An assessment/migration tool supported for the selected engine pair.

Implementation

57. Inventory engine version, size, objects, features, users, jobs, dependencies, growth, and downtime objective.

58. Run compatibility assessment and classify each finding as blocker, remediation, accepted difference, or test requirement.
59. Build the target with identity, private network plan, logging, backup, and capacity.
60. Perform an initial migration or load and validate schema, counts, hashes, aggregates, and permissions.
61. Generate source changes and, where supported, synchronize them online while measuring lag.
62. Execute a timed cutover rehearsal: freeze writes, final sync, switch connection, run smoke tests, and record elapsed time.
63. Apply a rollback decision threshold and demonstrate how source service would be restored if validation failed.

Validation Evidence

- Assessment findings have owners and closure evidence.
- Data validation covers business aggregates, not row count alone.
- Cutover duration and replication lag are measured.
- Rollback criteria are objective and executable.

Cleanup

- Delete migration services, target and source lab databases, temporary VMs, network resources, and logs not required as evidence.
- Revoke migration accounts and destroy temporary secrets.

Stretch challenge Run the rehearsal a second time after automation improvements and show the reduction in cutover duration and manual steps.

Interview Questions and Model Answers

1. What determines online versus offline migration?

Downtime tolerance, source/target support, data volume, change rate, network, complexity, and operational risk. Online migration reduces outage but adds replication and cutover complexity.

2. Why are row counts insufficient for validation?

Counts can match while values, constraints, encodings, permissions, sequences, time zones, or business aggregates are wrong.

3. What belongs in rollback criteria?

A measurable threshold such as final sync lag, failed critical smoke tests, data reconciliation variance, elapsed cutover time, or security control failure, plus an authorized decision owner.

4. Why rehearse with production-like data?

Data volume, distribution, large transactions, schema complexity, and workload concurrency determine migration duration and target performance.

Chapter Summary

- Migration begins with inventory, dependencies, assessment, baseline, and wave design.
- Online and offline modes have different complexity and downtime trade-offs.
- Validation spans data, function, performance, security, recovery, and operations.
- Cutover and rollback must be rehearsed, timed, automated, and objectively governed.

VOLUME 7C | CHAPTER 30

Chapter 30 - Data Security, Governance, Resilience, and Enterprise Capstone

Integrating identity, privacy, catalog, audit, backup, recovery, incident response, and platform ownership

Learning Objectives

- Apply defense in depth across data control planes, data planes, networks, keys, secrets, and audit evidence.
- Use Microsoft Purview, Defender, Azure Monitor, Policy, and platform-native controls for governance and detection.
- Design backup, restore, geo-recovery, retention, deletion, and incident procedures by data class.
- Integrate the full Part C service set into a reviewable enterprise data platform capstone.

30.1 Data Protection Is a Lifecycle

Data security begins before ingestion and continues through creation, use, sharing, archival, legal hold, and deletion. Classify data, minimize collection, assign owners, document lawful purpose and residency, define retention, and restrict access. Technical controls cannot compensate for data that was collected without a legitimate purpose or retained indefinitely without ownership.

Control domain	Required questions	Typical Azure controls
Identity and authorization	Who administers the resource? Who reads or changes data? How is privilege elevated and reviewed?	Entra groups, managed identities, PIM, Azure RBAC, database roles, Unity Catalog.
Network	Which paths can reach the service? Is public access necessary? How is DNS controlled?	Private Link, VNet integration, firewall, Private DNS, controlled egress.
Encryption and secrets	Who controls keys? How are rotations, backups, and recovery handled?	Service encryption, Key Vault, managed HSM where required, CMK, secretless identity.
Detection and audit	Which actions and threats must be visible, retained, and investigated?	Diagnostic settings, Log Analytics, Defender, SQL auditing, Cosmos diagnostics, Databricks audit logs.
Governance and lineage	Where is sensitive data? Who owns it? Which products consume it?	Microsoft Purview, classifications, lineage, catalog, labels, data contracts.
Resilience and deletion	How are corruption, outage, region failure, ransomware, and erasure handled?	PITR, long-term retention, geo replicas, continuous backup, immutable storage, tested deletion workflow.

30.2 Governance and Security Operations

Microsoft Purview can provide catalog, discovery, classification, lineage, and governance capabilities across data estates. Microsoft Defender services and platform threat-detection features add security findings, but findings must route into an owned triage process. Azure Policy can enforce diagnostic settings, private

endpoints, approved regions, tags, and public-network restrictions where supported. Exceptions must be time-bounded, justified, and reviewed.

30.3 Recovery Engineering by Failure Type

Failure	Primary mechanism	Important validation
Infrastructure or zone failure	Built-in HA or zone-redundant deployment.	Application reconnects; dependencies are also resilient.
Regional outage	Geo replica, failover group, multi-region Cosmos account, paired analytics design.	DNS/endpoints, identity, network, data loss, capacity, and runbook timing.
Accidental delete or corruption	Point-in-time restore, continuous backup, versioning, Delta history where safe.	Restore to separate target; business reconciliation; no overwrite of evidence.
Ransomware or malicious admin	Immutable/isolated copies, least privilege, PIM, alerting, break-glass, incident response.	Attacker cannot delete all recovery points; access is auditable.
Pipeline logic error	Quarantine, versioned tables, staged publish, replay from raw data.	Known-good source retained; rollback/rebuild produces correct result.
Privacy deletion request	Cataloged data locations and controlled deletion workflow.	Deletion propagates to replicas, caches, derived products, and backups according to policy/law.

30.4 Enterprise Data Platform Capstone

Scenario A regulated commerce company needs transactional ordering, customer profiles, low-latency catalog reads, real-time events, analytics, migration from legacy SQL Server, private connectivity, governed access, and tested regional recovery.

Target Architecture

- Azure SQL Database or SQL Managed Instance for transactional orders based on compatibility assessment, protected by zone resilience, backups, and failover design.
- Cosmos DB for globally distributed customer or catalog access where partition and consistency requirements justify it.
- Azure Managed Redis for bounded cache-aside acceleration with circuit breaking and datastore protection.
- Event Hubs and Schema Registry for governed business events, with Capture into ADLS Gen2.
- Data Factory for controlled batch/hybrid movement and migration orchestration.
- ADLS Gen2 plus Fabric, Synapse, or Azure Databricks selected through an architecture decision record for analytical products.
- Purview, Defender, Azure Monitor, Key Vault, Private Link, managed identities, RBAC, database roles, and immutable evidence across the platform.

Capstone Delivery Plan

64. Document business capabilities, data classes, owners, RPO/RTO, residency, access personas, latency, throughput, retention, and cost constraints.
65. Create architecture decision records for each data service, consistency level, partition key, cache pattern, lakehouse/warehouse engine, and migration mode.
66. Build the landing-zone controls: management group policy, resource groups, tags, budgets, identities, private DNS, monitoring, and Key Vault.

67. Deploy the transactional database and synthetic application; implement schema migration and failover-aware connection behavior.
68. Deploy Cosmos DB with tested partition key and consistency, then measure RU and hot-partition behavior.
69. Add Managed Redis cache-aside with TTL jitter, coalescing, circuit breaker, and failure test.
70. Publish versioned order events to Event Hubs, archive with Capture, and process through an idempotent consumer.
71. Ingest raw and operational data into the governed lake and build curated analytical tables with quality evidence.
72. Run a migration rehearsal from legacy SQL Server or PostgreSQL, validate data, and execute a controlled cutover simulation.
73. Perform incidents: database blocking, expired credential attempt, hot Cosmos partition, cache outage, consumer lag, bad pipeline release, and accidental data corruption.
74. Execute restore and regional-recovery drills, measuring achieved RPO/RTO and documenting gaps.
75. Present architecture, security, cost, observability, recovery evidence, runbooks, and a prioritized remediation backlog.

Capstone Acceptance Criteria

Area	Acceptance evidence
Architecture	Decision records connect requirements to service, model, consistency, partition, and recovery choices.
Security	No static credentials in code; private-access design; least privilege; audit and threat findings routed to owners.
Reliability	Restores and failovers are executed, timed, reconciled, and compared with objectives.
Data quality	Contracts, schema compatibility, quarantine, counts, freshness, and business reconciliation are measurable.
Operations	Dashboards, alerts, runbooks, owner, severity, escalation, and postincident actions exist.
Cost	Budgets, tags, capacity evidence, idle-resource cleanup, and cost allocation by product are demonstrated.
Delivery	Infrastructure, schema, pipeline, and job changes are source-controlled, tested, approved, and traceable.

Troubleshooting Matrix

Symptom	Likely cause	Investigation	Corrective action
Private data service inaccessible after policy rollout	Policy deployed private endpoint or disabled public access without DNS/routing readiness.	Review policy compliance, deployment logs, endpoint state, DNS resolution, routes, and client path.	Stage policy, repair DNS/network, test in canary scope, and use approved time-bounded exception if required.
Security alert has no owner	Diagnostics enabled but routing, severity, runbook, or service ownership missing.	Trace alert rule, action group/SIEM route, asset inventory, and on-call mapping.	Assign product owner, severity, response SLA, automation, and closure evidence.
Restore succeeds technically	Wrong restore point,	Compare timestamps, event	Select correct point, replay

Symptom	Likely cause	Investigation	Corrective action
but business totals differ	incomplete downstream rebuild, time-zone/window mismatch, or unprocessed events.	offsets, pipeline checkpoints, source totals, and derived tables.	events, rebuild products, and reconcile before traffic cutover.
Sensitive data appears in lower environment	Uncontrolled copy, missing masking, catalog/classification gap, or broad export permission.	Use audit logs, lineage, storage scans, identity history, and pipeline runs.	Contain access, rotate credentials, purge per procedure, use synthetic/masked data, and remediate governance controls.

Hands-on Lab 30: Execute the Enterprise Data Platform Capstone

Objective Integrate secure transactional, distributed, cache, streaming, lakehouse, migration, governance, and recovery patterns into one evidence-backed platform review.

Prerequisites

- Completion of representative labs from Chapters 21-29.
- An approved sandbox with strict budgets and synthetic data only.
- A small team or role simulation covering platform, database, data engineering, security, and application ownership.

Implementation

76. Create the requirements, data classification, RPO/RTO, architecture diagram, and decision records.
77. Deploy a reduced-cost version of the target architecture using at least one relational service, Event Hubs, ADLS Gen2, and one analytics engine.
78. Use managed identities, group-based authorization, private-access planning, logging, and centralized secrets.
79. Implement a restartable batch pipeline and an idempotent event consumer.
80. Run a migration rehearsal and business-level data reconciliation.
81. Inject three failures and follow runbooks, collecting detection, diagnosis, mitigation, and recovery timelines.
82. Restore corrupted data to a separate target and perform a planned failover or regional tabletop exercise.
83. Produce the final operational evidence pack and prioritized improvements.

Validation Evidence

- Architecture decisions are traceable to measurable requirements.
- No lab component depends on an undocumented individual credential or manual step.
- Data quality, event replay, restore, and failure evidence is reproducible.
- Cost and cleanup reports show all temporary resources are removed.

Cleanup

- Decommission all capstone resources in dependency order and retain only approved evidence.
- Revoke temporary roles, credentials, private DNS links, endpoints, replicas, caches, clusters, and monitoring resources.

Stretch challenge Repeat the capstone as a 90-minute architecture and incident interview: present trade-offs, receive a failure scenario, diagnose it, and defend the recovery and cost plan.

Interview Questions and Model Answers

1. How do you choose between Azure SQL, PostgreSQL, Cosmos DB, and a lakehouse?

Start from data model, transactions, access patterns, consistency, latency, scale, analytics, compatibility, recovery, and team skills. Often a platform uses several services with explicit ownership boundaries.

2. What is the most important control for data recovery?

A tested restore or failover that proves business correctness. Backup configuration without restoration and reconciliation evidence is only an assumption.

3. How do managed identities improve data security?

They remove application-managed credentials and allow Azure resources to acquire scoped tokens. They still require least-privilege role design, monitoring, and lifecycle control.

4. What should a data platform SLO include?

User-facing availability or freshness, latency, quality, recovery, and sometimes processing completeness - measured at the data-product boundary, not only service uptime.

5. How do you prevent a data lake from becoming a data swamp?

Define zones, contracts, owners, quality, catalog, lineage, access, lifecycle, cost allocation, and promotion criteria, then enforce them through platform automation and review.

Chapter Summary

- Data security spans identity, network, encryption, audit, governance, recovery, privacy, and deletion.
- Purview, Defender, Policy, and Monitor require ownership and response processes.
- Recovery must be designed per failure type and validated at business level.
- The capstone integrates service selection, migration, streaming, analytics, security, operations, and cost evidence.

Part C Command Reference

Area	Example command or query	Purpose
Azure SQL	<code>az sql db show -g <rg> -s <server> -n <db></code>	Inspect SQL database configuration.
PostgreSQL	<code>az postgres flexible-server show -g <rg> -n <server></code>	Inspect Flexible Server state and HA.
MySQL	<code>az mysql flexible-server show -g <rg> -n <server></code>	Inspect MySQL Flexible Server.
Cosmos DB	<code>az cosmosdb show -g <rg> -n <account></code>	Inspect regions and consistency.
Managed Redis	<code>az redisenterprise show -g <rg> -n <cache></code>	Inspect Azure Managed Redis resource.
Storage	<code>az storage account show -g <rg> -n <account></code>	Inspect ADLS/Blob account settings.
Data Factory	<code>az datafactory pipeline list -g <rg> --factory-name <name></code>	List factory pipelines.
Event Hubs	<code>az eventhubs eventhub show -g <rg> --namespace-name <ns> -n <hub></code>	Inspect partitions and retention.
Databricks	<code>databricks jobs list</code>	List jobs through Databricks CLI.
Migration	<code>az datamigration service list -g <rg></code>	List Database Migration Service resources.
Diagnostics	<code>az monitor diagnostic-settings list --resource <id></code>	Verify logs and metrics routing.
Cost cleanup	<code>az resource list -g <rg> -o table</code>	Review remaining lab resources.

Production Readiness Checklist

- Requirements include transaction, consistency, latency, throughput, data growth, RPO/RTO, residency, retention, privacy, and cost.
- Every data product has an owner, contract, schema/version policy, quality metrics, freshness objective, and support path.
- Managed identities and group-based authorization replace static credentials where supported.
- Private connectivity, DNS, egress, firewall, and administrative access are tested from every required client path.
- Backups, continuous backups, replicas, failover groups, checkpoints, and event retention have been restored or replayed.
- Query, partition, connection, RU, cache, lag, pipeline, file, cluster, quality, and cost metrics are observable.
- Schema and pipeline changes are source-controlled, tested, backward compatible, approved, and traceable.
- Alerts have severity, owner, route, runbook, response target, and closure evidence.
- Data classification, catalog, lineage, access review, retention, deletion, and legal requirements are implemented.
- Capacity and cost tests use representative volume and concurrency; idle resources terminate automatically.

- Failure tests cover database unavailability, corruption, cache outage, event lag, pipeline replay, regional outage, and credential compromise.
- Cutover, rollback, restore, failover, and privacy-deletion procedures have objective acceptance criteria.

Cross-Service Troubleshooting Method

84. Define the user-visible symptom, affected data product, start time, and business impact.
85. Identify the transaction or data path across application, identity, DNS, network, database/cache, event stream, pipeline, lakehouse, and consumer.
86. Correlate deployment, schema, policy, key/secret, network, capacity, and workload changes.
87. Inspect platform health and service limits, then product-level SLOs, data quality, lag, and recovery state.
88. Use logs, metrics, traces, query diagnostics, offsets, checkpoints, lineage, and audit records to test hypotheses.
89. Mitigate safely with load shedding, failover, restore, replay, rollback, or feature control without destroying evidence.
90. Reconcile business data after mitigation and record timeline, root cause, contributing controls, and corrective actions.

Glossary

Term	Meaning
PITR	Point-in-time restore from managed backup history.
Failover group	Cross-region SQL grouping with coordinated failover and listener endpoints.
RPO	Maximum acceptable data loss measured in time.
RTO	Maximum acceptable time to restore service.
Partition key	Field used to distribute and route Cosmos DB data and requests.
Request unit	Normalized measure of Cosmos DB operation cost.
Consumer group	Independent Event Hubs offset/checkpoint view for one logical consumer application.
Schema Registry	Central repository for event serialization schemas.
ADLS Gen2	Azure Data Lake Storage with hierarchical namespace over Blob Storage.
OneLake	Microsoft Fabric unified logical data lake.
Delta Lake	Table protocol and transaction log over data-lake files.
Unity Catalog	Azure Databricks unified governance layer for data and AI assets.
Integration Runtime	Data Factory compute/network bridge for movement and activity dispatch.
Watermark	Persisted boundary identifying the last completed incremental data interval.
Idempotency	Property that allows repeated processing without repeated business side effects.

Term	Meaning
Data contract	Documented schema, semantics, quality, freshness, ownership, and compatibility expectations.
Lineage	Record of how data moves and transforms from source to products.
Hot partition/key	A distribution unit receiving disproportionate traffic or storage.
Cache stampede	Many callers simultaneously regenerate the same expired or missing cached value.
Cutover	Controlled transition of production traffic or writes from source to target system.

Official Microsoft Documentation Index

Azure SQL failover groups: <https://learn.microsoft.com/azure/azure-sql/database/failover-group-sql-db>

Azure SQL business continuity: <https://learn.microsoft.com/azure/azure-sql/database/business-continuity-high-availability-disaster-recover-hadr-overview>

PostgreSQL reliability: <https://learn.microsoft.com/azure/reliability/reliability-database-postgresql>

PostgreSQL high availability: <https://learn.microsoft.com/azure/postgresql/high-availability/concepts-high-availability>

Cosmos DB consistency: <https://learn.microsoft.com/azure/cosmos-db/consistency-levels>

Cosmos DB global distribution: <https://learn.microsoft.com/azure/cosmos-db/global-distribution>

Azure Cache for Redis retirement FAQ: <https://learn.microsoft.com/azure/azure-cache-for-redis/retirement-faq>

Azure Data Lake Storage Gen2: <https://learn.microsoft.com/azure/storage/blobs/data-lake-storage-introduction>

OneLake overview: <https://learn.microsoft.com/fabric/onelake/onelake-overview>

Data Factory managed virtual network: <https://learn.microsoft.com/azure/data-factory/managed-virtual-network-private-endpoint>

Data Factory integration runtime: <https://learn.microsoft.com/azure/data-factory/concepts-integration-runtime>

Event Hubs overview: <https://learn.microsoft.com/azure/event-hubs/event-hubs-about>

Event Hubs Schema Registry: <https://learn.microsoft.com/azure/event-hubs/schema-registry-overview>

Azure Databricks introduction: <https://learn.microsoft.com/azure/databricks/introduction/>

Unity Catalog: <https://learn.microsoft.com/azure/databricks/data-governance/unity-catalog/>

Azure Database Migration Service: <https://learn.microsoft.com/azure/dms/dms-overview>

Microsoft Purview: <https://learn.microsoft.com/purview/>

Defender for SQL: <https://learn.microsoft.com/azure/defender-for-cloud/defender-for-sql-introduction>

Completion standard You have completed Volume 7C when you can justify a data-service choice from workload semantics, deploy it securely, measure its behavior, migrate or replay data safely, restore it, diagnose failures with evidence, and remove every temporary resource.